

PERFORMANCE PROFILING AND OPTIMIZATION USING GPROF AND PERF

**INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SRI CITY,
CHITTOOR**

HIGH PERFORMANCE COMPUTING LAB

Lab 01

Name: Ch. Sai Ganesh

Roll Number: S20240010049

Semester: UG-3 (Semester 5)

Department: Computer Science and Engineering

Course

High Performance Computing

Aim

The aim of this experiment is to study the performance of different sorting algorithms using Linux performance analysis tools. In this experiment, Bubble Sort and Quick Sort are implemented and their execution times are compared. The experiment also uses GPROF to identify the functions consuming maximum CPU time and studies the effect of compiler optimizations on program performance.

Introduction

Performance analysis plays an important role in High Performance Computing. Even though two programs may produce the same output, their execution times may differ significantly because of the algorithms used and compiler optimizations.

Sorting is one of the most common operations performed in computer science. Different sorting algorithms have different time complexities and therefore show different performance when the size of the input increases.

In this experiment, two sorting algorithms, namely Bubble Sort and Quick Sort, are implemented. Bubble Sort is considered as the baseline algorithm because of its simplicity, whereas Quick Sort is used as an efficient sorting algorithm.

The performance of both algorithms is analyzed using Linux tools such as **GPROF**. GPROF identifies the functions consuming the maximum CPU time, helping the programmer locate the bottlenecks in the application. The experiment also studies compiler optimizations using GCC optimization levels -O0, -O2 and -O3.

Bubble Sort

Bubble Sort is one of the simplest sorting algorithms. It repeatedly compares two adjacent elements and swaps them whenever they are in the wrong order.

During every iteration, the largest element moves towards the end of the array, similar to how an air bubble rises to the surface of water.

Although Bubble Sort is easy to understand and implement, it is inefficient for large datasets because it performs a large number of comparisons and swaps.

Advantages

- Easy to implement.
- Suitable for learning sorting concepts.
- Requires no additional memory.

Disadvantages

- Very slow for large datasets.
 - Large number of unnecessary comparisons.
 - Time complexity is $O(n^2)$.
-

Quick Sort

Quick Sort is a Divide and Conquer algorithm.

It selects a pivot element and partitions the array into two smaller sub-arrays. Elements smaller than the pivot are moved to the left, whereas larger elements are moved to the right. The same procedure is recursively applied until the entire array becomes sorted.

Quick Sort is one of the fastest sorting algorithms used in practice.

Advantages

- Much faster than Bubble Sort.
- Efficient for large datasets.
- Average time complexity is $O(n \log n)$.

Disadvantages

- Worst case complexity is $O(n^2)$.
 - Uses recursion.
-

GPROF

GNU GPROF is a profiling tool available in Linux. It helps programmers identify the functions consuming the maximum execution time.

Instead of guessing which function is slow, GPROF provides detailed statistics such as:

- Percentage of CPU time consumed.
- Number of function calls.

- Function call graph.
- Time spent inside each function.

These statistics help programmers optimize the application effectively.

PERF

PERF is a Linux performance analysis tool used to collect hardware-level statistics such as CPU cycles, instructions executed, cache references, cache misses and branch misses.

During this experiment, PERF could not be executed because the program was developed in Windows Subsystem for Linux (WSL2). The Microsoft WSL2 kernel used in the system does not provide the required Linux performance tools.

This limitation is specific to the execution environment and not to the program itself.

Algorithm

Bubble Sort Algorithm

1. Generate random input data.
2. Compare adjacent elements of the array.
3. If the first element is greater than the second, swap them.
4. Continue this process until the largest element reaches the last position.
5. Repeat the same process for the remaining unsorted elements.
6. Continue until the entire array is sorted.

Time Complexity

Case	Complexity
Best Case	$O(n)$

Average Case	$O(n^2)$
-----------------	----------

Worst Case	$O(n^2)$
------------	----------

Quick Sort Algorithm

1. Select the last element as the pivot.
2. Partition the array into two parts.
3. Place all smaller elements before the pivot.
4. Place all larger elements after the pivot.
5. Recursively apply Quick Sort to the left sub-array.
6. Recursively apply Quick Sort to the right sub-array.
7. Continue until the complete array is sorted.

Time Complexity

Case	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$

Program Description

The program was implemented in the C programming language. It generates random integer values representing temperature data. Three arrays are created:

- Original array
- Bubble Sort array
- Quick Sort array

The original array is copied into two separate arrays so that both sorting algorithms work on the same input data. This ensures a fair comparison.

The execution time of each sorting algorithm is measured using the `clock()` function available in the C standard library. After sorting, the program prints the execution time of Bubble Sort and Quick Sort.

The program also includes functions for swapping elements, partitioning the array during Quick Sort, and recursively sorting the partitions.

The source code consists of the following functions:

- `swap()`
- `bubbleSort()`
- `partition()`
- `quickSort()`
- `main()`

The implementation generates random data, copies it into two arrays, sorts them independently, and measures execution time.

Compilation and Execution

The program was compiled using GCC under Ubuntu Linux.

Without Compiler Optimization

```
gcc -O0 -pg mysort.c -o mysort
```

Run the program

```
./mysort
```

Measure execution time

```
time ./mysort
```

Generate GPROF report

```
gprof mysort gmon.out > myreport.txt
```

With -O2 Optimization

```
gcc -O2 -pg mysort.c -o mysort_O2
```

```
./mysort_O2
```

```
gprof mysort_O2 gmon.out > report_O2.txt
```

With -O3 Optimization

```
gcc -O3 -pg mysort.c -o mysort_O3
```

```
./mysort_O3
```

```
gprof mysort_O3 gmon.out > report_O3.txt
```

Results

The execution times obtained during the experiment are shown below.

Optimization	Bubble Sort	Quick Sort
-O0	0.3500 sec (approx.)	0.0040 sec

-O2	0.1040 sec	0.0000 sec
-O3	0.2629 sec	0.0041 sec

(Replace the -O0 values with the exact values from your `time` output if you have them.)

GPROF Observations

The GPROF report generated for the program clearly shows that the `bubbleSort()` function consumed the highest percentage of CPU time. In the `-O0` report, Bubble Sort accounted for 80% of the sampled execution time, while the `swap()` function consumed the remaining 20%. Quick Sort consumed very little profiling time because it completed much faster.

For the optimized versions (`-O2` and `-O3`), `bubbleSort()` remained the hotspot function and accounted for nearly 100% of the sampled execution time.

Screenshots

(Insert your screenshots here in this order.)

- Running the program
 - `time ./mysort`
 - `myreport.txt`
 - `report_02.txt`
 - `report_03.txt`
-

Performance Comparison

Parameter	Bubble Sort	Quick Sort
-----------	-------------	------------

Algorithm Type	Iterative	Divide and Conquer
Average Time Complexity	$O(n^2)$	$O(n \log n)$
Number of Comparisons	Very High	Comparatively Low
Execution Speed	Slow	Fast
Suitable for Large Data	No	Yes
GPROF Hotspot	Yes	No

Discussion

Bubble Sort required a significantly larger execution time because every element is compared repeatedly with adjacent elements. The number of comparisons increases rapidly with the size of the input.

Quick Sort divides the array into smaller sub-arrays and recursively sorts them. Due to this divide-and-conquer approach, Quick Sort completed much faster.

The compiler optimization levels also influenced the execution time. Optimized executables generally required fewer instructions and produced more efficient machine code.

Answers to Assignment Questions

1. Which sorting algorithm performed better? Explain your observation.

Quick Sort performed significantly better than Bubble Sort because its average time complexity is $O(n \log n)$, whereas Bubble Sort has $O(n^2)$ complexity. As the number of elements increases, Bubble Sort becomes considerably slower, while Quick Sort continues to perform efficiently.

2. Which function consumed the maximum execution time?

The GPROF report identified `bubbleSort()` as the hotspot function. It consumed the highest percentage of CPU time in all profiling reports.

3. How does execution time change with increasing input size?

The execution time of Bubble Sort increases rapidly because the algorithm performs approximately n^2 comparisons. Quick Sort scales much better since its average complexity is $O(n \log n)$, making it suitable for larger datasets.

4. How did compiler optimizations (-O2 and -O3) improve performance?

Compiler optimizations improve performance by reducing unnecessary instructions, improving register allocation, performing function inlining where appropriate, and optimizing loops. These optimizations generally reduce execution time and improve efficiency.

5. Which algorithm would you recommend for large datasets?

Quick Sort is recommended because it executes much faster than Bubble Sort for large datasets and has a better average time complexity. It is widely used in practical applications due to its efficiency.

Conclusion

This experiment compared the performance of Bubble Sort and Quick Sort using Linux profiling tools. Both algorithms successfully sorted the generated data; however, Quick Sort consistently outperformed Bubble Sort in terms of execution time.

GPROF analysis showed that the `bubbleSort()` function consumed the majority of the execution time, identifying it as the primary performance bottleneck. Compiler optimization levels also affected the execution speed, demonstrating the importance of optimization during software development.

The PERF analysis could not be completed because the experiment was performed using Windows Subsystem for Linux (WSL2), where the required Linux performance tools were unavailable for the Microsoft kernel.

Overall, the experiment demonstrates that selecting an efficient algorithm and applying suitable compiler optimizations can significantly improve program performance.